Playbook — StopCriStudio

Guida di riferimento del progetto

Descrizione del progetto

Stop Cri Studio è un'applicazione web che serve a scrivere e organizzare le descrizioni delle API. Un'API, o interfaccia di programmazione, è il modo in cui due programmi si scambiano informazioni: per esempio l'app del meteo che chiede i dati a un server. La descrizione di un'API è un documento che elenca gli indirizzi disponibili, le richieste accettate e le risposte possibili. Il formato usato qui è OpenAPI, uno standard aperto e molto diffuso.

Il programma è pensato per chi progetta API REST: sviluppatori, studenti, analisti e piccoli team. Serve a chi vuole un editor visuale semplice, senza installare nulla di pesante e senza creare un account.

Il motivo per cui esiste è l'offline. Stop Cri Studio gira interamente nel browser. Non manda i dati a un server, non richiede una connessione e non chiede credenziali. Ogni modifica viene salvata in automatico nella memoria locale del browser. Il risultato può essere esportato in un file YAML da consegnare, versionare o pubblicare.

Cosa fa il progetto

All'apertura il programma mostra una schermata di benvenuto con tre scelte: creare una nuova specifica, caricare una specifica già salvata, oppure importare un file OpenAPI esistente in formato YAML o JSON. Da qui si entra sempre nell'editor visuale.

Nell'editor si può: - Compilare la scheda informativa dell'API, con nome e versione della specifica. - Aggiungere percorsi, cioè indirizzi come /utenti oppure /ordini/{id}. - Aggiungere operazioni a ogni percorso, scegliendo il metodo HTTP: GET per leggere, POST per creare, PUT e PATCH per modificare, DELETE per cancellare, e così via. - Gestire i parametri di percorso, quelli scritti tra parentesi graffe nell'indirizzo, assegnando a ciascuno un tipo e una descrizione. - Aggiungere parametri di query, cioè i valori che seguono il punto interrogativo in un indirizzo. Possono essere semplici, oggetti o liste, con vincoli come valore minimo, massimo, espressione regolare e valore predefinito. - Definire il corpo della richiesta per le operazioni POST, PUT e PATCH, scegliendo il tipo di contenuto e allegando uno schema scritto sul momento oppure un riferimento a uno schema riutilizzabile. - Aggiungere risposte con un codice di stato, una descrizione e, se serve, uno schema che mostra la struttura dei dati restituiti. - Impostare i requisiti di sicurezza di ogni operazione, per esempio una chiave API da mettere nell'intestazione, oppure l'autenticazione HTTP di base o con token. - Creare e gestire schemi riutilizzabili, cioè modelli di dati con un nome, da usare in più punti della specifica. - Filtrare l'elenco dei percorsi e degli schemi dalla barra laterale per ritrovarli in fretta.

L'importazione accetta solo file con estensione .yaml, .yml o .json. Prima di aprire il documento, il programma controlla che i campi obbligatori ci siano e che la versione OpenAPI sia la 3.0.x o la 3.1.x. Se qualcosa non va, mostra una finestra con l'elenco degli errori e propone di scegliere un altro file.

L'esportazione produce un file YAML. Durante l'esportazione il programma completa la specifica con alcuni elementi richiesti dalle linee guida italiane per le API: genera un identificatore univoco per ogni operazione, assicura che ogni operazione abbia almeno un'etichetta e almeno una risposta valida. Il nome del file viene ricavato dal nome della specifica.

Il salvataggio è automatico: circa un secondo dopo l'ultima modifica, la specifica viene scritta nella memoria locale del browser. Alcune modifiche

vengono salvate subito, senza attendere. La schermata di caricamento mostra tutte le specifiche salvate e permette di riaprirle o eliminarle.

Concetti chiave

Specifica OpenAPI: documento che descrive un'API in modo leggibile sia da una persona sia da un programma. Contiene informazioni generali, percorsi, operazioni, parametri, risposte, schemi e requisiti di sicurezza. In questo progetto la specifica viene gestita come un oggetto con un identificatore, un nome, una versione, una data di creazione, una data di ultima modifica e il contenuto completo.

API REST: modo di organizzare un servizio web intorno a risorse, cioè oggetti come utenti, ordini o animali, raggiungibili tramite indirizzi. Ogni indirizzo si chiama percorso.

Percorso: l'indirizzo di una risorsa, per esempio /utenti oppure /ordini/{id}. Le parti tra parentesi graffe sono segnaposto e si chiamano parametri di percorso.

Operazione: l'azione compiuta su un percorso, identificata dal metodo HTTP. GET legge, POST crea, PUT sostituisce, PATCH modifica in parte, DELETE cancella. Un percorso può avere più operazioni, ma al massimo una per metodo.

Parametro di percorso: valore obbligatorio inserito nell'indirizzo al posto di un segnaposto. Per esempio in /ordini/{id} il parametro id potrebbe valere 42. Il programma controlla che il nome sia formato solo da lettere, numeri e trattini bassi, e che non ci siano due segnaposto attaccati come {id}{nome}.

Parametro di query: valore facoltativo o obbligatorio aggiunto in fondo all'indirizzo dopo un punto interrogativo, per esempio /cerca?q=libri. Può essere semplice, un oggetto con più proprietà, oppure una lista. Può avere vincoli come valore minimo e massimo, espressione regolare e valore predefinito.

Corpo della richiesta: il contenuto che il client invia al server, per esempio i dati di un nuovo ordine. In OpenAPI esiste solo per alcuni metodi, tipicamente POST, PUT e PATCH. Il programma permette di scegliere il tipo di contenuto, per esempio application/json, e di allegare uno schema.

Risposta: ciò che il server restituisce dopo una richiesta. Ogni risposta ha un codice di stato, come 200 per successo, 201 per creazione, 404 per

risorsa non trovata, e una descrizione. Può avere anche uno schema che descrive la struttura dei dati.

Schema: modello di dati. Descrive i campi di un oggetto, il tipo di ciascun campo, quali campi sono obbligatori e quali no. Uno schema riutilizzabile ha un nome e vive in una sezione apposita della specifica, chiamata components.schemas. Altri punti della specifica possono richiamarlo con un riferimento, scritto come #/components/schemas/NomeSchema. Il simbolo $ref indica proprio un riferimento a uno schema.

Requisito di sicurezza: regola che dice come un'operazione deve essere protetta. I tipi supportati sono la chiave API, che può viaggiare in un'intestazione, in un parametro di query o in un cookie, e l'autenticazione HTTP di base o con token di tipo bearer. Ogni operazione può richiedere uno o più schemi di sicurezza.

Tipo di contenuto: etichetta che dice in che formato sono i dati, per esempio application/json oppure multipart/form-data.

YAML e JSON: due formati di testo per scrivere dati in modo strutturato. YAML è più leggibile per le persone, JSON è più diffuso tra i programmi. Il programma legge entrambi e scrive YAML.

Memoria locale del browser: piccola area di archiviazione dentro il browser, dove il programma salva le specifiche. I dati restano su quel computer e non vengono inviati altrove. Se si svuota la memoria del browser, le specifiche salvate si perdono.

Autosalvataggio: scrittura automatica delle modifiche nella memoria locale, circa un secondo dopo l'ultima modifica. Serve a evitare di perdere il lavoro se si chiude la pagina per errore.

operationId e tag: due informazioni che le linee guida italiane per le API richiedono per ogni operazione. L'operationId è un nome univoco ricavato dal metodo e dal percorso, per esempio getUsersById. Il tag è un'etichetta che raggruppa le operazioni affini.

Versione della specifica e versione OpenAPI: sono due cose diverse. La versione della specifica è il numero scelto dall'autore, per esempio 1.0.0 o 2.3.4. La versione OpenAPI è la versione dello standard usato, fissata a 3.0.0 per le nuove specifiche e accettata come 3.0.x o 3.1.x in importazione. Nel programma la versione OpenAPI è indicata separatamente.

Formato interno ed esportazione: mentre si lavora, il programma tiene alcune informazioni in una forma comoda per l'editor, per esempio i parametri di query in un elenco dedicato e il corpo della richiesta in una struttura separata. Al momento dell'esportazione queste informazioni vengono trasformate nel formato OpenAPI standard. L'utente non deve preoccuparsene: vede sempre e solo il risultato finale pulito.

Il dominio

Il dominio è la descrizione delle API REST. Un esempio realistico: un negozio di animali online vuole pubblicare l'API del proprio catalogo. Il team scrive una specifica OpenAPI che dice: c'è un percorso /pets per leggere e creare animali; c'è un percorso /pets/{petId} per leggere, modificare o cancellare un singolo animale; il parametro petId è un numero intero ed è obbligatorio; l'operazione GET su /pets restituisce una lista di oggetti Pet, ciascuno con id numerico e name testuale; l'operazione POST su /pets accetta un corpo in application/json con i dati del nuovo animale e risponde con il codice 201; ogni operazione richiede una chiave API nell'intestazione X-API-Key.

Tutto questo si può costruire passo passo nell'editor. I percorsi si aggiungono nella sezione Paths. Le operazioni si aggiungono scegliendo il metodo. I parametri di percorso compaiono automaticamente quando si scrive {petId} nell'indirizzo. I parametri di query si aggiungono con un modulo che chiede nome, tipo, obbligatorietà e vincoli. Il corpo della richiesta si definisce scegliendo il tipo di contenuto e aggiungendo le proprietà. Le risposte si aggiungono con codice, descrizione e schema. I requisiti di sicurezza si collegano a uno schema definito nella sezione Security.

Alla fine l'esportazione produce un file YAML simile a questo:

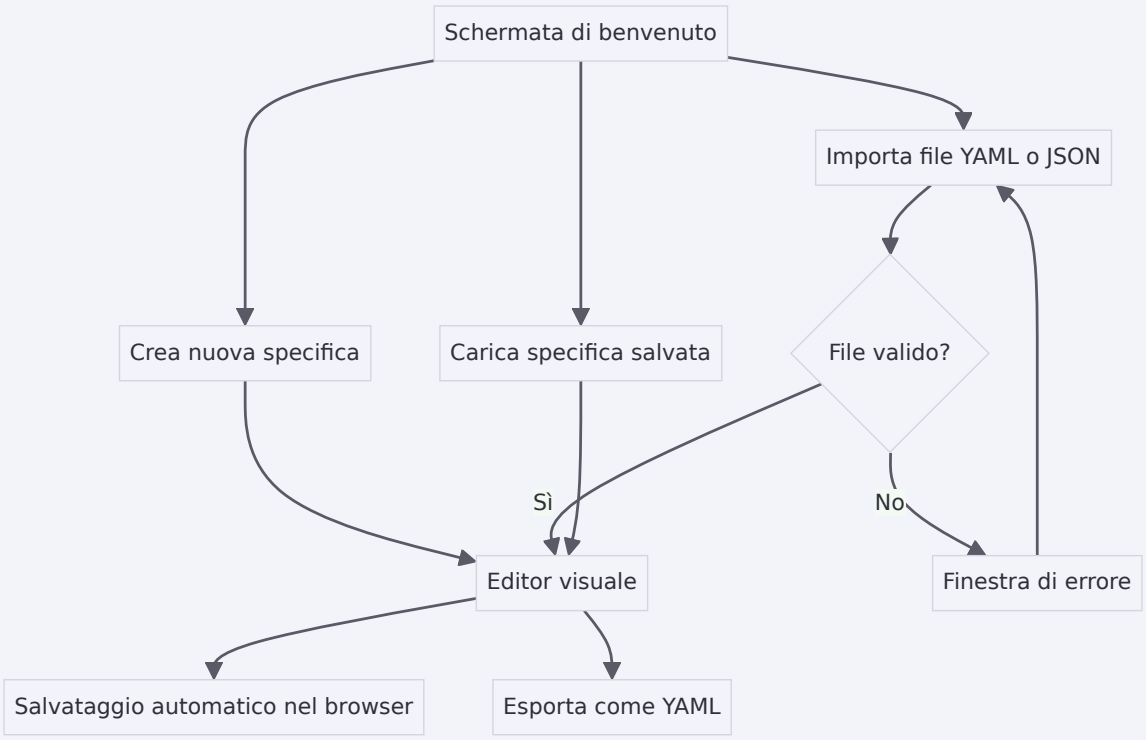
openapi: 3.0.0 info: title: Negozio di animali version: 1.0.0 paths: /pets: get: operationId: getPets tags: - animali responses: '200': description: Elenco degli animali /pets/{petId}: get: operationId: getPetsByPetId parameters: - name: petId in: path required: true schema: type: integer

Questo è il linguaggio che il resto del mondo usa per capire e consumare l'API. Un altro programmatore può leggere il file e generare un client, della documentazione o dei test automatici.

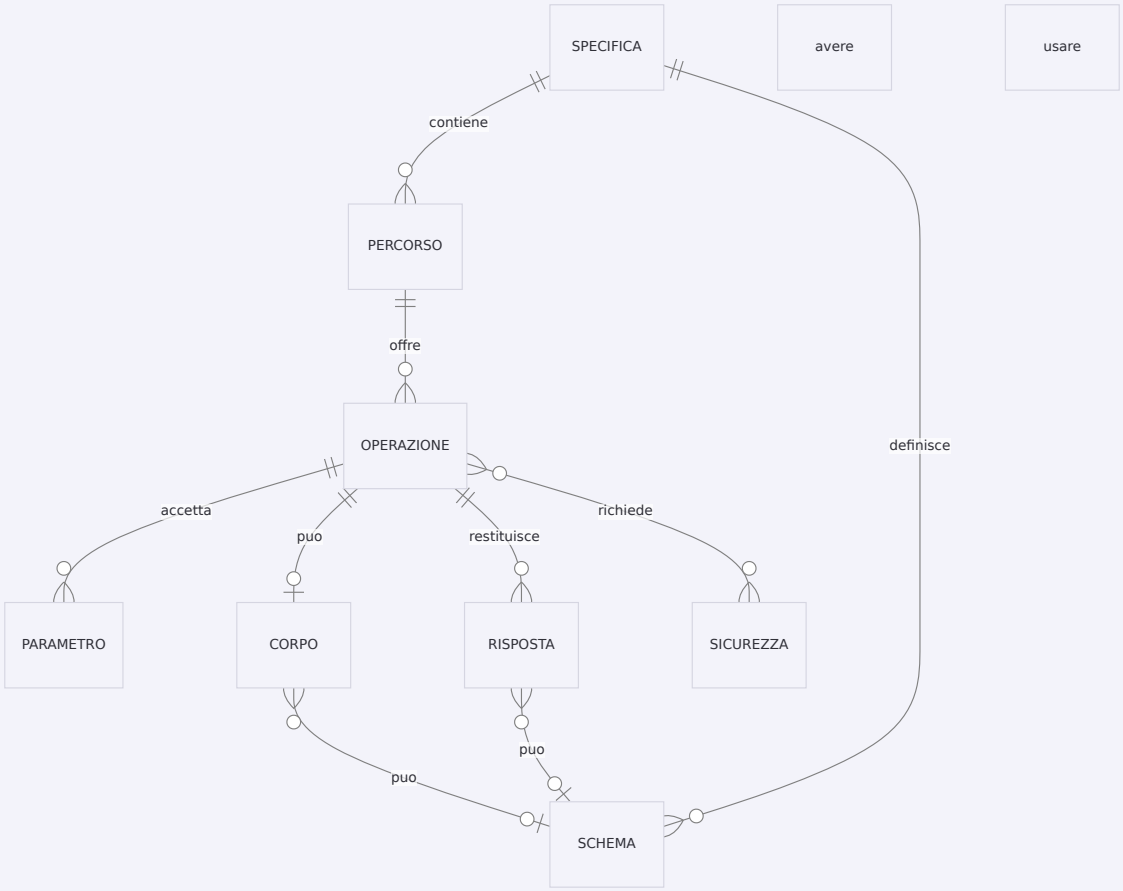
Un secondo scenario tipico è la manutenzione. Un'API già esistente viene descritta in un file YAML o JSON prodotto da un altro strumento. Il team lo importa, ritrova i percorsi nell'editor, corregge una descrizione, aggiunge un parametro di query dimenticato e riesporta. In entrambi i casi il valore è lo stesso: passare da un'idea confusa a un documento ordinato, verificabile e condivisibile.

Lavorare offline ha un senso concreto. Molte aziende non vogliono caricare su servizi esterni i progetti di API ancora riservati. Molte aule non hanno una

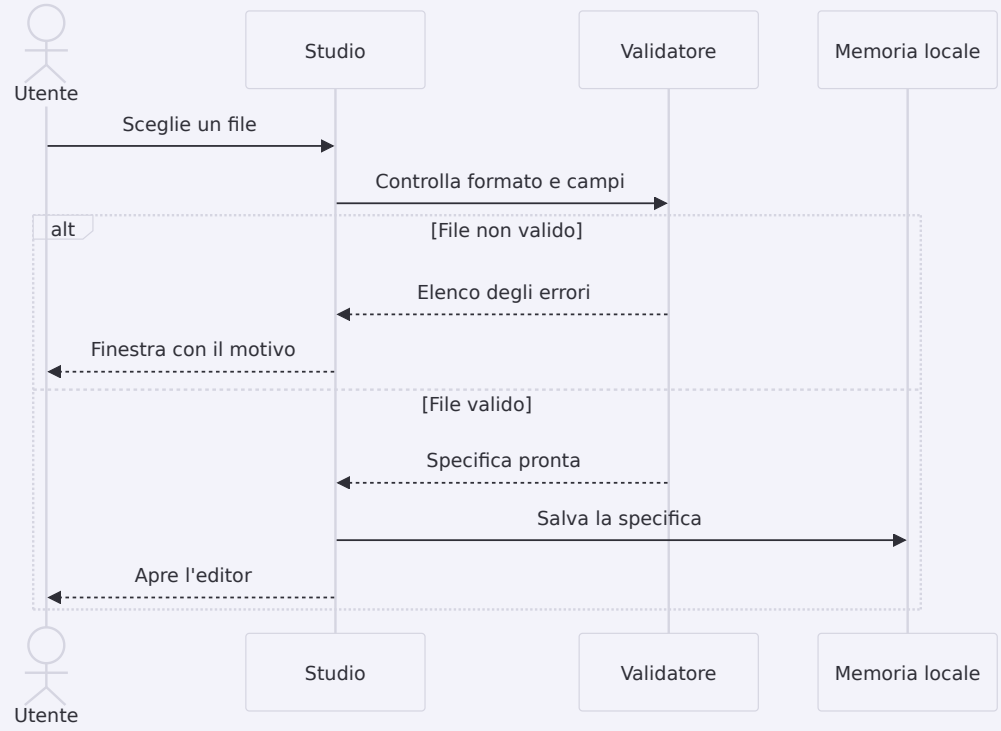
connessione stabile. Molti studenti non vogliono creare account. Stop Cri Studio toglie tutti e tre questi ostacoli: la specifica vive nel browser, si salva da sola e si sposta solo quando l'utente decide di esportarla.



Il viaggio di un utente: dal benvenuto all'editor, fino al salvataggio locale e all'esportazione.



Le entità della specifica e i loro legami: una specifica contiene percorsi, ogni percorso offre operazioni, ogni operazione può avere parametri, un corpo, risposte e requisiti di sicurezza, e schemi riutilizzabili.



Che cosa succede quando si importa un file: il controllo avviene prima di aprire l'editor, e solo un file valido viene salvato.

Casi d'uso

La studentessa che consegna un progetto. Giulia deve descrivere l'API di una biblioteca scolastica. Non ha una connessione stabile e non vuole creare account. Apre Stop Cri Studio, sceglie Crea, inserisce nome e versione, aggiunge tre percorsi con le relative operazioni, esporta il file YAML e lo consegna. Esito: un file pronto, senza aver installato nulla.

Lo sviluppatore che eredita una specifica. Marco riceve da un fornitore un file YAML con l'API di un servizio. Lo importa, il programma controlla i campi obbligatori e lo apre nell'editor. Marco aggiunge un parametro di query mancante, corregge una descrizione e riesporta il file. Esito: la specifica aggiornata, senza riscrivere tutto da zero.

Il file sbagliato. Luca prova a importare un file di testo qualsiasi. Il programma rifiuta l'estensione e mostra una finestra con il motivo e un pulsante per scegliere un altro file. Esito: Luca capisce subito che cosa serve e riprova con il file giusto.

La libreria locale del team. Sara ha cinque specifiche salvate nel browser. Apre la schermata di caricamento, ritrova quella giusta dall'elenco con nome e versione, la riapre e continua a lavorare. Quando un vecchio esperimento non serve più, lo elimina dopo una conferma. Esito: la sua libreria resta ordinata.

Lo schema riutilizzabile. Chiara sta definendo il corpo di una richiesta POST come oggetto scritto sul momento. Si accorge che la stessa struttura servirà in altre operazioni. Con un comando dedicato crea uno schema riutilizzabile con un nome, e il corpo della richiesta passa a usare un riferimento. Esito: un solo modello da aggiornare, invece di tante copie.

Il lavoro non perso. Durante una sessione lunga, Andrea chiude il browser per errore. Riapre Stop Cri Studio e ritrova la specifica nell'elenco dei salvataggi locali, con le ultime modifiche già scritte. Esito: nessun lavoro perso.